Chapter 14 File Processing

14.1 Introduction

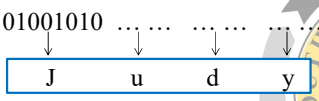
- Information Storage
  - Temporary Data
    - Variables, Arrays, Consts, …
    - Store in Memory
    - Little
    - Fast access
  - Persistent Data
    - Files(文件)
    - Store on Secondary storage device (e.g.: magnetic disks, optical disks, tapes…)
    - Large amounts of data
    - Slowly access

14.2 Data Hierarchy

- Bit (binary digit)
  - A digital that can assume one of two values, `0` or `1`
- Character
  - Decimal, letters and special symbols(e.g.: @, %, &,…)
  - C++ provide data type `char` (occupy one byte of memory) and `wchar_t` (occupy more than one byte memory)

14.2 Data Hierarchy(cont.)

- Field
  - A group of characters that conveys some meaning (represented as data member in C++)
- Record
  - Composed of several related fields(represented as a class in C++)
  - A record-key is a field unique to each record



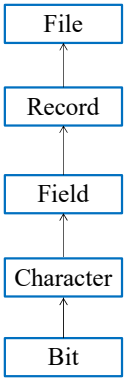
	Account	Last_Name	First_Name	Balance
key ⟶	001	Green	Judy	10000.00

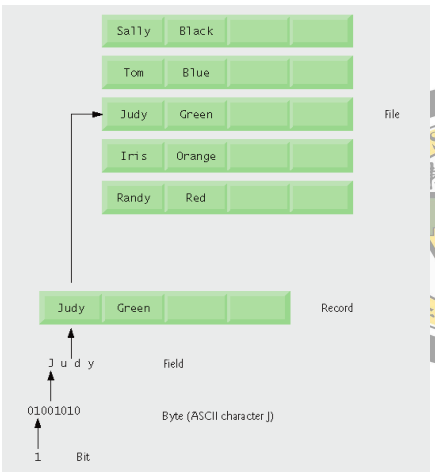
14.2 Data Hierarchy(cont.)

- File
  - Records typically are stored in order by a record-key field

Account	Last_Name	First_Name	Balance
001	Green	Judy	10,000.00
002	Smith	John	2,000.00
…	…	…	…
100	Li	Bob	10.00

14.2 Data Hierarchy(cont.)





14.2 Data Hierarchy(cont.)

- Category of Files (*By storage format*)
  - Text file (文本文件)
    - 文本文件以字节（byte）为单位，每字节对应一个ASCII码，表示一个字符，故又称字符文件或者ASCII文件；
    - 文本文件保存的是一串ASCII字符，可用文字处理器对其进行编辑（如：可以直接送到显示器或者打印机上显示或打印出对应的字符，供人们直接阅读），输入输出过程中系统要对内、外存的数据格式进行相应转换；

14.2 Data Hierarchy(cont.)

- Category of Files (*By storage format*)
  - Binary file (二进制文件)
    - 二进制文件以位（bit）为单位，它的内容是数据的计算机内部表示，实际上也就是由0和1组成的序列；
    - 输入输出过程中，系统不对相应数据进行任何转换，故又称内部格式文件；

14.2 Data Hierarchy(cont.)

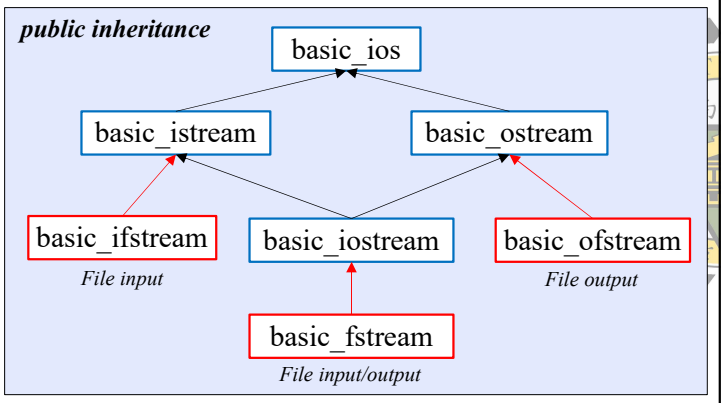
- Category of Files (*By accessing way*)
  - Sequential file(顺序存取文件)

    只能顺序的查找记录
  - Random-access file(随机存取文件)

    可以直接定位记录，不需顺序的查找

14.3 Files and Streams(cont.)

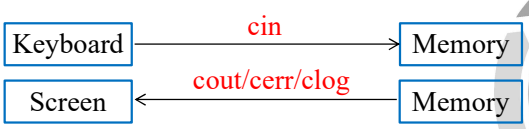
- I/O stream template hierarchy



14.3 Files and Streams(cont.)

- I/O stream objects(I/O流对象)

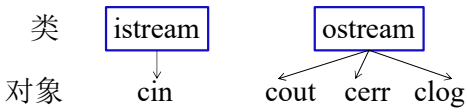
(1) Standard I/O stream objects (标准I/O流对象)



- 标准I/O流类库在std命名空间中预定义了一批标准I/O流对象（cin、cout、cerr、clog），提供内存与常用外部设备（键盘、屏幕）进行数据交互功能。
- 定义在头文件<iostream>中。

14.3 Files and Streams(cont.)

- 标准I/O流对象



cin ： 标准输入设备(通常为键盘) → 内存

cout： 内存 → 标准输出设备(通常为显示器)

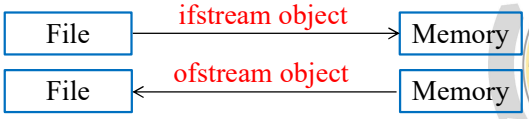
cerr： 内存 → 标准错误设备，无缓冲，立即显示

clog： 内存 → 标准错误设备，有缓冲，缓冲区满或清空时输出

14.3 Files and Streams(cont.)

- I/O stream objects(I/O流对象)

(2) File I/O stream objects (文件I/O流对象)



- 操作文件时，需自定义ifstream /ofstream流对象，以指定所具体操作的文件和操作相关的参数。
- 定义在头文件<fstream>中。

14.3 Files and Streams(cont.)

- Communication between a program and a file is performed through stream objects
  - `#include <iostream>`
    `#include <fstream>`
  - `<fstream>`
    - includes the definition of three class templates:
      - » `basic_ifstream` — for file input
      - » `basic_ofstream` — for file output
      - » `basic_fstream` — for file input and output
    - provides template specialization that enable `char` I/O
      - » `ifstream`— enables `char` input from file (*read*)
      - » `ofstream`— enables `char` output to file (*write*)
      - » `fstream`— enables `char` input from, and output to file

14.4 Creating a Sequential File(cont.)

- Sequential File(顺序文件)

Account `int`	Last_Name `string`	First_Name `string`	Balance `double`
001	Green	Judy	10,000.00
002	Smith	John	2,000.00
...	…	…	…
100	Li	Bob	10.00

001 Green Judy 10000.00 002 Smith John 2000.00 100 Li Bob 10.00

record1 record2 record3

Lengths of records are different.
Can just be accessed sequentially.

14.4 Creating a Sequential File(cont.)

- Create an `ofstream` object

(1) Opening a file by creating stream object
(创建流类对象的同时打开文件)

```
ofstream ( const char* filename, int mode );
```

- filename: path + file name(postfix)
  - ➢ `"c:\\clients.dat"`
  - ➢ `"clients.dat"`

14.4 Creating a Sequential File(cont.)

- mode:
  - ➢ `using std::ios;`
  - ➢ `ios::out` // default mode in ofstream
    ① if the file already exists, then open it and discard all data
    ② if the file does not exist, then first create it
  - ➢ `ios::app` // append data to the end of the file

- Example:

```
Ofstream outClientFile("clients.dat")
```


14.4 Creating a Sequential File(cont.)

(2) First create stream object, then open the file
(先创建流对象, 后打开文件)

- Default constructor + `open()`
- `void open( const char* filename, int mode);`
- Example:

```
ofstream outClientFile;
outClientFile.open("clients.dat")
```

14.4 Creating a Sequential File(cont.)

- File open mode (文件打开模式)

文件打开方式	含义
ios::in	以输入(读)方式打开文件
ios::out	以输出(写)方式打开文件
ios::app	打开一个文件使新的内容始终添加在文件的末尾
ios::ate	打开一个文件使新的内容添加在文件尾，但下次添加时，写在当前位置处
ios::trunc	若文件存在，则清除文件所有内容；若文件不存在，则创建新文件
ios::binary	以二进制方式打开文件，缺省时以文本方式打开文件
ios::nocreate	打开一个已有文件，若该文件不存在，则打开失败
ios::noreplace	若打开的文件已经存在，则打开失败

以ios::out模式打开时，默认是ios::trunc

14.4 Creating a Sequential File(cont.)

- Example:

Fig 14.4: Output text into a sequential file (将数据写入顺序文件)

14.4 Creating a Sequential File(cont.)

- `ios` operators (常用操作一：判断文件是否成功打开)
  - Overloaded Operator `operator!`

```
if( !outClientFile )
{
   cerr << " file can not be opened! ";
   exit(1);
}
```

  - Judge whether or not file be successfully open
    - `true`: unsuccessful (打开不存在的文件、没有读写权限、写时空间不够)
    - `false`: successful

14.4 Creating a Sequential File(cont.)

- `ios` operators (常用操作二：判断文件是否读取结束)
  - Overloaded Operator `operator void*`

```
while( cin >> account >> name >> balance )
{
   ……
}
```

```
while( !cin.eof() )
{
   ……
}
```


14.4 Creating a Sequential File(cont.)

- `ios` operators (常用操作二：判断文件是否读取结束)

```
while( cin >> account >> name >> balance )
{
   ……
}
```

如果用作条件判断，则隐式强制进行`void*`类型的类型转换：

- 如果输入成功：非空指针
- 如果输入失败：空指针

再将指针隐式的转换为`bool`类型：

- 非空指针：`true`
- 如果输入失败：`false`

14.4 Creating a Sequential File(cont.)

- `ios` operators (常用操作三)
  - Write data into a file (向文件写入数据)

```
outClientFile << myVariable;
```

14.4 Creating a Sequential File(cont.)

- `ios` operators (常用操作三)
  - Write data into a file (向文件写入数据)

```
outClientFile << myVariable;
```

  - Close file (关闭文件)
    - Implicitly: Invoke `outClientFile` destructor
    - Explicitly:

```
outClientFile.close();
```

    - Performance tips(17.1):
      Close file explicitly when the program no longer need to reference them.

14.5 Reading Data from a Sequential File

- Create an `ifstream` object
  - (1) Opening a file by creating stream object
    (创建流类对象的同时打开文件)
    - `ios::in` // default mode in ifstream

```
ifstream inClientFile("client.dat",ios::in);
```

  - (2) First create stream object, then open the file
    (先创建流对象, 后打开文件)

```
ifstream inClientFile;
inClientFile.open("clients.dat")
```


14.5 Reading Data from a Sequential File(cont.)

- `ios` operators
  - Overloaded Operator `operator!`

```
!outClientFile
```

    - Judge whether or not the file be successfully open
  - Overloaded Operator `operator void*`

```
while ( inClientFile >> myVariable )
```

    - Judge whether or not meet EOF of the file


```
1  // Fig. 14.7: Fig14_07.cpp
3  #include <iostream>
13 #include <fstream>
16 #include <iomanip>
20 #include <string>
23 #include <cstdlib>
24 using namespace std;
25
26 void outputLine( int, const string, double ); // prototype
27
28 int main()
29 {
30    // ifstream constructor opens the file
31    ifstream inClientFile( "clients.dat", ios::in );
32
33    // exit program if ifstream could not open file
34    if ( !inClientFile )
35    {
36       cerr << "File could not be opened" << endl;
37       exit( 1 );
38    }
39
40    int account;
41    char name[ 30 ];
42    double balance;
```

Open `clients.dat` for input

Overloaded `operator!` returns `true` if `clients.dat` was not opened successfully

```
44    cout << left << setw( 10 ) << "Account" << setw( 13 )
45       << "Name" << "Balance" << endl << fixed << showpoint;
46
47    // display each record in file
48    while ( inClientFile >> account >> name >> balance )
49       outputLine( account, name, balance );
50
51    return 0; // ifstream destructor closes the file
52 }
53
54 // display single record from file
55 void outputLine( int account, const string
56 {
57    cout << left << setw( 10 ) << account << setw( 13 ) << name
58       << setw( 7 ) << setprecision( 2 ) <<
59 } // end function outputLine
```

Overloaded `operator void *` returns a null pointer `0` (`false`) when the end of `clients.dat` is reached

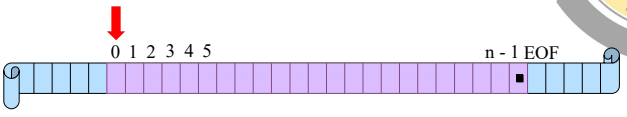
`ifstream` destructor implicitly closes the file

```
Account    Name          Balance
100        Jones           24.98
200        Doe            345.67
300        White            0.00
400        Stone          -42.16
500        Rich           224.62
```

以ASCII码形式从文本文件读取数据，格式化的输出

14.5 Reading Data from a Sequential File(cont.)

- File-position pointer (文件定位指针)
  - Number of the next byte to be read("`get`" pointer) or written("`put`" pointer)
  - Question:
    - How to relocate file-position pointer?



14.5 Reading Data from a Sequential File(cont.)

- Member functions for pointer relocation (指针重定位函数)
  - `istream` member functions
    - `seekg( streamoff, ios::seek_dir );`
    - `tellg();` // return the current position of "get" pointer as type `long`

14.5 Reading Data from a Sequential File(cont.)

- Member functions for pointer relocation (指针重定位函数)
  - `ostream` member functions
    - `seekp( streamoff, ios::seek_dir );`
    - `tellp();` // return the current position of "put" pointer as type `long`
  - Two arguments:
    - offset: number of bytes from the pointer to the specified location (as type long)

14.5 Reading Data from a Sequential File(cont.)

- direction: seeking direction
  - `ios::beg` – Position relative to the beginning (默认)
  - `ios::cur` – Position relative to the current position
  - `ios::end` – Position relative to the end

14.5 Reading Data from a Sequential File(cont.)

- Examples
  - `fileObject.seekg( 0 );`
    - » position to the 0th byte of fileObject (default by ios::beg)
  - `fileObject.seekg( n );`
    - » position to the nth byte of fileObject
  - `fileObject.seekg( n, ios::cur );`
    - » position n bytes forward in fileObject
  - `fileObject.seekg( y, ios::end );`
    - » position y bytes back from end of fileObject
  - `fileObject.seekg( 0, ios::end );`
    - » position at end of fileObject
  - `location = fileObject.tellg();`
    - » assigns the "get" pointer value to variable location

- 文件读写
  - 顺序文件操作：只能从文件的开始处依次顺序读写文件内容，不能任意读写文件内容。
  - 读：文件流类的`get()`、`getline()`、`read()`成员函数以及提取符 ">>"
  - 写：`put()`、`write()`函数以及插入符 "<<"

函数原型	说 明
get(char &ch)	从文件中读取一个字符
getline(char *pch,int count,char delim='\n')	从文件中读取多个字符，读取个数有参数count决定，参数delim是读取字符是指定的结束符（默认为换行字符）
read(char *pch,int count)	从文件中读取多个字符，读取个数由参数counnt决定
put(char ch)	向文件写入一个字符
write(const char *pch,int count)	向文件写入多个字符，字符个数由count决定

14.5 Reading Data from a Sequential File(cont.)

- `get()`
  - `int istream::get();//提取一个字符,包括空格,制表,垂直制表、换页、换行和回车等，与cin有所不同。注意返回为整型。`
  - `istream&istream::get(char &);`
  - `istream&istream::get(unsigned char &);//提取一个字符,放在字符型变量中。单参数，并为字符的引用。`
  - 使用举例：`cin.get();`

14.5 Reading Data from a Sequential File(cont.)

- getline()
  - <string>

    istream& getline (istream &is , string & str, char delim)//第一个参数为 istream类对象，一般为 cin， 第二个参数为 string 类对象，第三个参数默认为 '\n'
  - <iostream>

    cin.getline(char*array,int count)//第一个参数为 char* 类型，指向一个字符串；第二个参数为 int 类型，表示读取一行中的前 count-1 个字符
  - 使用举例：cin.getline(arrayname,strnum);

读写顺序文件举例

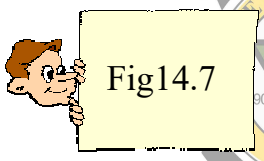
```
#include <iostream>
using std::cerr;
using std::cout;
using std::endl;
using std::ios;

#include <fstream>
using std::ofstream;
using std::ifstream;

#include <cstdlib>
#include <cstring>

int main(){
   ofstream fout;
   fout.open("c:\\output.txt");
   if (!fout) {
      cerr<<"File could not be
      opened"<<endl;
      exit(1);
   }
   int num = 150;
   char name[] = "John Doe";
   fout << num << '\n';
   fout << name << '\n';
   fout.close();
```

```
   ifstream fin("c::\\output.txt");
   int number;
   char name2[20];
   fin >> number; fin.ignore();
   fin.getline(name2,'\n');
   //fin>>number>>name2;
    cout<<number<<"  "<< name2<<endl;
   strcpy(name2,"aaaa  bbb");
   cout<<"now name2="<<name2<<endl;
   fin.seekg(5);
   fin.getline(name2,'\n');
   cout<<"new name2  ="<< name2 <<
   endl;
   return 0;
}
```

```
150  John Doe
now name2=aaaa  bbb
new name2=John Doe
Press any key to continue
```


14.5 Reading Data from a Sequential File(cont.)

- 贷款查询程序
  - Zero balance: 没有消费, 没有存款
    - 1 balance == 0
  - Credit balance: 有存款
    - 2 balance < 0
  - Debit balance: 有欠款
    - 3 balance > 0



- Fig14.7/line65: inClientFile.clear();
  // reset EOF for next input
  line66: inClientFile.seekg(0)
  // relocate to beginning of file